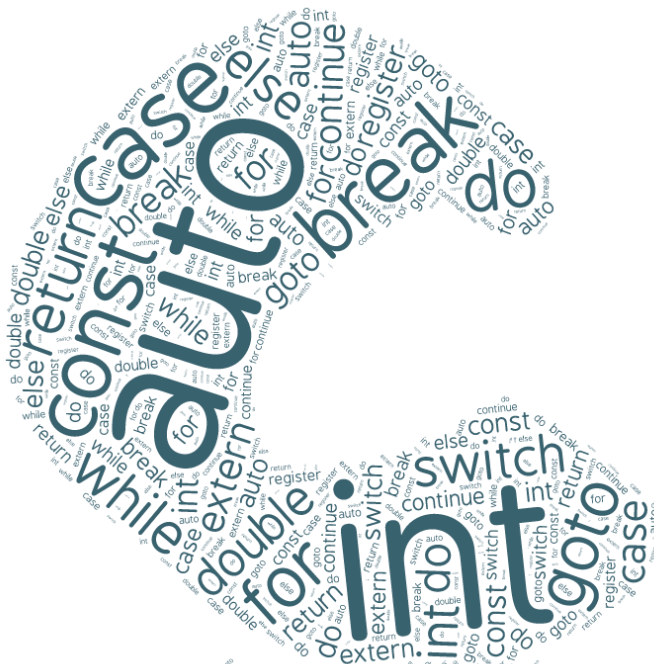




chapter 6

指针 (续1)

1

指针的基本概念

2

指针运算

3

指针与数组

4

指针数组与多级指针

5

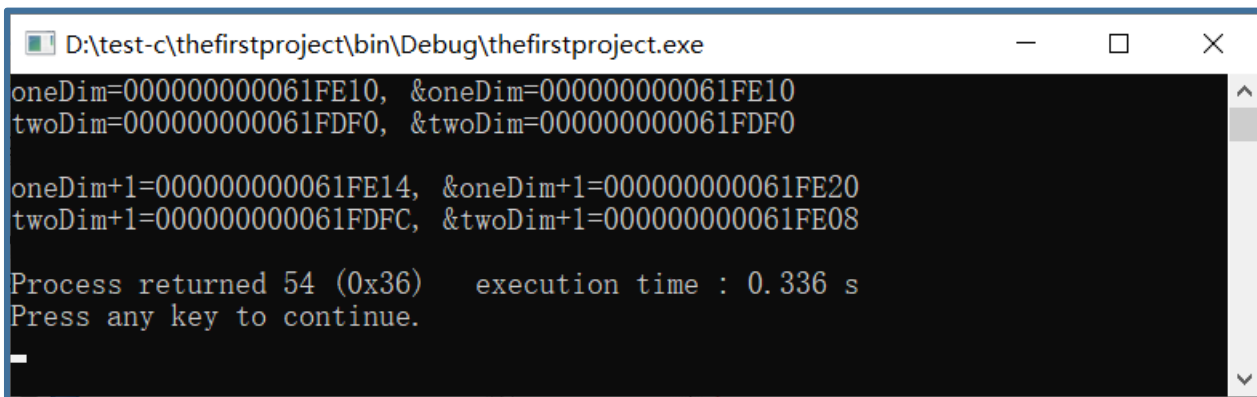
指针与函数

6

综合应用

数组名a代表数组首元素的起始地址，&a才是整个数组的首地址

```
1  #include <stdio.h>
2  void main()
3  {
4      int oneDim[4]={1,2, 3, 4};
5      int twoDim[2][3]={1,2,3,4,5,6};
6      printf("oneDim=%p, &oneDim=%p\n", oneDim, &oneDim);
7      printf("twoDim=%p, &twoDim=%p\n", twoDim, &twoDim);
8      printf("\n");
9      printf("oneDim+1=%p, &oneDim+1=%p\n", oneDim+1, &oneDim+1);
10     printf("twoDim+1=%p, &twoDim+1=%p\n", twoDim+1, &twoDim+1);
11 }
```



```
D:\test-c\thefirstproject\bin\Debug\thefirstproject.exe
oneDim=000000000061FE10, &oneDim=000000000061FE10
twoDim=000000000061FDF0, &twoDim=000000000061FDF0

oneDim+1=000000000061FE14, &oneDim+1=000000000061FE20
twoDim+1=000000000061FDFC, &twoDim+1=000000000061FE08

Process returned 54 (0x36)    execution time : 0.336 s
Press any key to continue.
```

数组与指针在访问内存时采用统一的地址计算方法，都可以处理内存中连续存放的一系列数据，两者之间既有密切关系，又有本质区别。

6.3.1 一维数组与指针

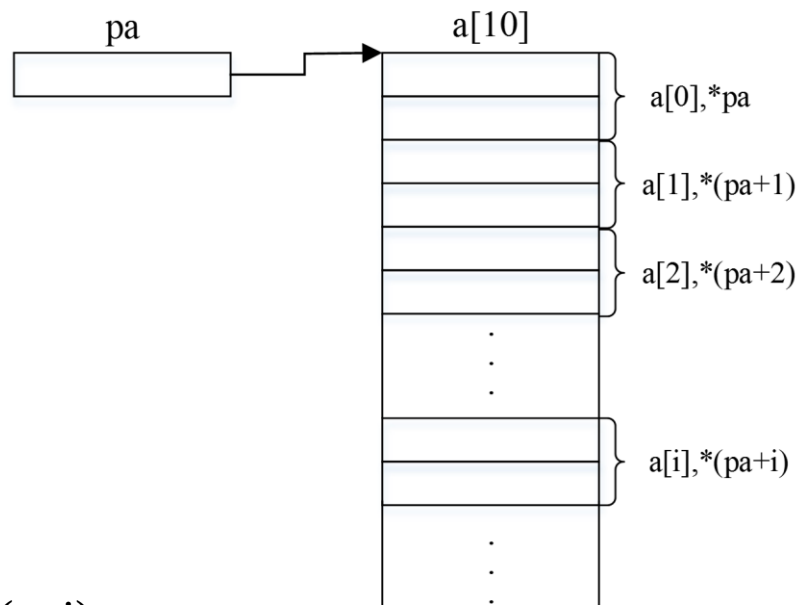
一维数组与指针的关系

```
int a[10];  
int *pa;  
pa=a; 或 pa=&a[0];
```

$*pa \longleftrightarrow a[0]$

$*(pa+1) \longleftrightarrow a[1]$

$*(pa+i) \longleftrightarrow a[i] \longleftrightarrow pa[i] \longleftrightarrow *(a+i)$



四种形式

例6.4 指针和数组表现形式互换性的程序

```
#include <stdio.h>
void main( )
{
    int i,*pa;
    int a[]={2,4,6,8,10};

    pa=a;

    for (i=0 ; i<5 ; i++)
        printf ("a[%d]:%d ", i ,pa[i]);

    printf ("\n");

    for(i=0 ; i<5 ; i++)
        printf ("*(pa+%d):%d ", i,*(a+i));

    printf ("\n");
}
```

指针指向数组
的首地址

指针采用数组
的形式使用

数组采用指针
的形式使用

指针和数组的本质不同：指
针是地址变量，而数组名是
地址常量

```
pa = a;
pa++,p--;
pa += n;
```

```
a = pa;
a++, a--;
a += n;
```



运行结果：

```
a[0]:2   a[1]:4   a[2]:6   a[3]:8   a[4]:10
*(pa+0):2 *(pa+1):4 *(pa+2):6 *(pa+3):8
*(pa+4):10
```

例6.4 将数组a中n个整数按相反的顺序存放

```
#include <stdio.h>
void invert(int *pdata,int n); //函数原型声明
void main( )
{
    int i;
    int data[10]={1,80,2,5,8,12,45,56,9,6};
    int *p=data; //定义指针变量p
    printf("original array:\n");
    for(i=0;i<10;i++) printf("%-4d",*(p+i));
    printf("\n");

    invert(p,10);
    printf("inverted array:\n");
    for(i=0;i<10;i++)
        printf("%-4d",p[i]);
}

void invert(int *pdata, int n)
{
    int i,j,temp;
    for(i=0,j=n-1;i<j;i++,j--)
    {
        temp=pdata[i];
        pdata[i]=pdata[j];
        pdata[j]=temp;
    }
}
```

调用函数(以指针作为实参)

一维数组的信息一般要包含数组地址和元素个数两个参数

可替代为: void invert(int pdata[],int n)

分析: 将a[0]与a[n-1]对换, 再将a[1]与a[n-2]对换.....

运行结果:

original array:

1 80 2 5 8 12 45 56 9 6

inverted array:

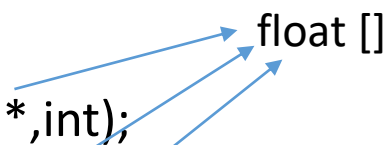
6 9 56 45 12 8 5 2 80 1

例6.9 编写三个函数分别完成指定一维数组元素的数据输入、求一维数组的平均值、求一维数组的最大值和最小值。由主函数完成这些函数的调用。

分析：采用**地址传递**的方式，把一维数组的存储**首地址**作为实参数调用函数。在被调用的函数中，**以指针变量作为形式参数接收数组的地址**。该指针被赋予数组的地址之后，使用这个指针就可以对数组中的所有数据进行处理。

函数原型：

```
void input(float *,int);  
float average(float *,int);  
void maxmin(float *,int, float *,float *);
```



最大值和最小值需要返回，采用地址传递方式

6.3.1

一维数组与指针

例6.9 编写三个函数分别完成指定一维数组元素的数据输入、求一维数组的平均值、求一维数组的最大值和最小值。由主函数完成这些函数的调用。

函数原型声明

```
#include <stdio.h>

void input(float *, int);
float average(float *, int);
void maxmin(float *, int, float *, float *);
void main( )
{
    float data[10]; //一维数组定义
    float aver,max,min;
    float *num=data;
    input(data,10);
    aver=average(data,10);
    maxmin(num,10,&max,&min);
    printf("aver=%f\n",aver);    //输出平均值
    printf("max=%f,min=%f\n",max,min);
}

float average(float *pdata, int n)    //数组平均值函数
{
    int i;
    float avg;
    for(avg=0,i=0;i<n;i++)    avg+= pdata[i];
    avg /=n;                //求平均值
    return(avg);            //将平均值返回给被调用函数
}
```

求最大值和最小值，以指针num以及max与min的地址作为函数实参

```
void input(float *pdata,int n) //输入数据函数
{
    int i;
    printf("please input array data: ");
    for(i=0;i<n;i++) //逐个输入数组的数据
        scanf("%f",pdata+i);
}

void maxmin(float *pdata,int n, float *pmax, float *pmin)
{
    int i;
    *pmax=*pmin=pdata[0];
    for(i=1;i<n;i++)
    {
        if(*pmax<ndata[i]) //求最大值
            *pmax=ndata[i];
        if(*pmin>ndata[i]) //求最小值
            *pmin=ndata[i];
    }
}
```

指针形式

数组形式

运行结果：

```
please input array data:
10 2 5 9 8 7 22 51 15 12
aver=14.100000
max=51.000000,min=2.000000
```


- (1) 虽然定义数组时指定它包含N个元素，并用指针变量p指向某一数组元素，但是实际上指针变量p可以指向数组以后的存储单元，结果不可预期，应避免出现这样的情况。
- (2) 指向数组元素的指针变量也可以带下标，如p[i]。**p[i]被处理成*(p+i)**，如果p是指向一个整型数组元素a[0]，则p[i]代表a[i]。但是必须弄清楚p的当前值是什么？**如果当前p指向a[3]，则p[2]并不代表a[2]，而是a[3+2]，即a[5]。**

设p开始时指向数组a的首元素（即p=a）：

① p++; //使p指向下一元素a[1]
*p; //得到下一个元素a[1]的值

② *p++; /*由于++和*同优先级，结合方向自右而左，因此它等价于*(p++)。先引用p的值，实现*p的运算，然后再使p自增1*/

③ *(p++); //先取*p值，然后使p加1
*(++p); //先使p加1，再取*p

④ ++(*p); /*表示p所指向的元素值加1，如果p=a，则相当于++a[0]，若a[0]的值为3，则a[0]的值为4。注意：是元素a[0]的值加1，而不是指针p的值加1*/

⑤ 如果p当前指向a数组中第i个元素a[i]，则：

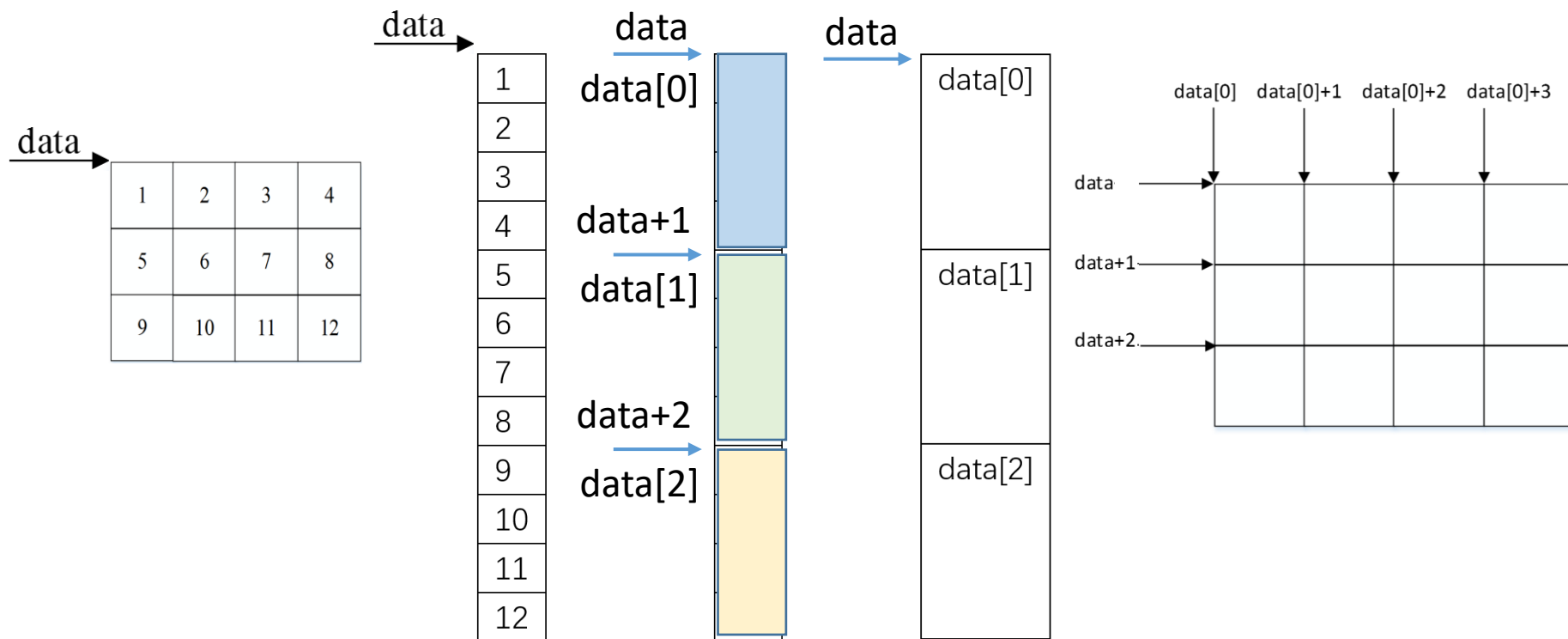
(p--) //相当于a[i--]，先对p进行“”运算，再使p自减

(++p) //相当于a[++i]，先使p自加，再进行“”运算

(--p) //相当于a[--i]，先使p自减，再进行“”运算

多维数组的地址： `int data[3][4]={1,2,3,4},{5,6,7,8},{9,10,11,12}};`

C语言中，多维数组被看成以**下一级数组**为元素的一维数组。



第一级分解，数组`data`被看成长度为3的一维数组

第二级分解，`data[0]`，`data[1]`，`data[2]`又是三个长度为4的一维数组

```
int a[3][4]={{1,3,5,7},{9,11,13,15},{17,19,21,23}};
```

		a[0]	a[0]+1	a[0]+2	a[0]+3
a	a[0]	a[0][0]	a[0][1]	a[0][2]	a[0][3]
		2000	2004	2008	2012
a+1	a[1]	1	3	5	7
		2016	2020	2024	2028
a+2	a[2]	9	11	13	15
		2032	2036	2040	2044
		17	19	21	23

表示形式	含义	地址
a	二维数组名，指向一维数组a[0]，即0行起始地址	2000
a[0], *(a+0), *a	0行0列元素地址	2000
a+1, &a[1]	指向第1行起始地址	2016
a[1], *(a+1)	1行0列元素a[1][0]的地址	2016
a[1]+2, *(a+1)+2, &a[1][2]	1行2列元素a[1][2]的地址	2024
*(a[1]+2), **(*a+1)+2, a[1][2]	1行2列元素a[1][2]的值	是元素值，为13

C语言的地址信息中既包含位置信息(如内存编号2000)，还包含它所指向的数据的类型信息。

a[0]是一维数组名，它是一维数组中起始元素的地址，**a**是二维数组名，它是二维数组的**首行**起始地址，二者的纯地址是相同的，即2000，但它们的**基类型不同**，即它们指向的数据的类型不同，前者是整型数据，后者是一维数组。

6.3.2

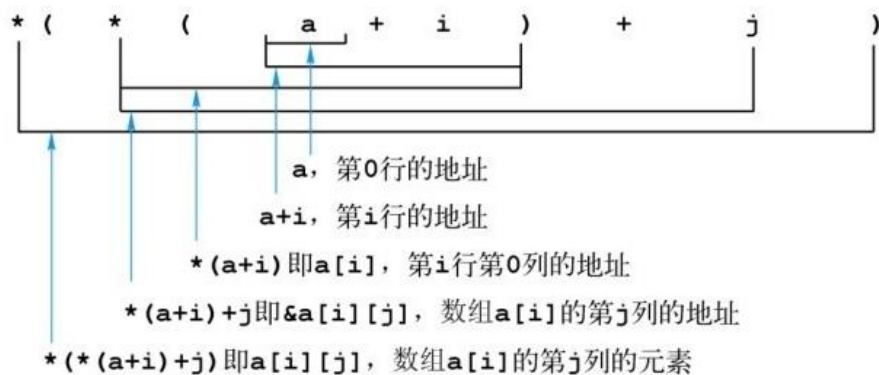
多维数组与指针

```
#include <stdio.h>
int main()
{
    int a[3][4]={1,3,5,7,9,11,13,15,17,19,21,23};
    printf("%d,%d\n",a,*a);           //0行起始地址和0行0列元素地址
    printf("%d,%d\n",a[0],*(a+0));    //0行0列元素地址
    printf("%d,%d\n",&a[0],&a[0][0]); //0行起始地址和0行0列元素地址
    printf("%d,%d\n",a[1],a+1);       //1行0列元素地址和1行起始地址
    printf("%d,%d\n",&a[1][0],*(a+1)+0); //1行0列元素地址
    printf("%d,%d\n",a[2],*(a+2));    //2行0列元素地址
    printf("%d,%d\n",&a[2],a+2);     //2行起始地址
    printf("%d,%d\n",a[1][0],*(*(a+1)+0)); //1行0列元素的值
    printf("%d,%d\n",*a[2],*(*(a+2)+0)); //2行0列元素的值
    return 0;
}
```

D:\Codeblocks\test\bin\Debug\test.exe

```
6422000, 6422000
6422000, 6422000
6422000, 6422000
6422016, 6422016
6422016, 6422016
6422032, 6422032
6422032, 6422032
9, 9
17, 17
```

```
Process returned 0 (0x0)   execution time : 0.147 s
Press any key to continue.
```



数组指针:

如何用单个指针变量处理多维数组?

单个指针变量可
处理一维数组



N维数组是以N-1维数组
为元素的一维数组



以N-1维数组为数据类型定义指针变量, 则
可处理以N-1维数组为元素的一维数组

数组指针的一般定义格式为:

<存储类型> <数据类型> (*数组指针名) [元素个数];

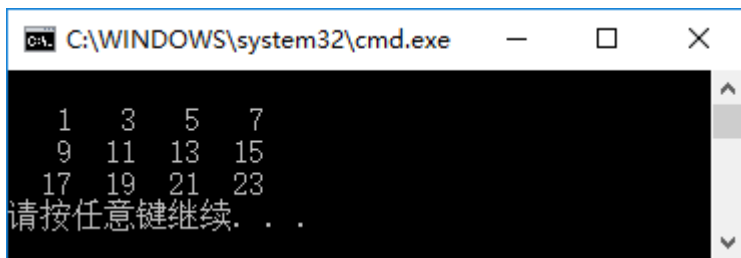
`int (*pt)[4];` //表示pt指向由4个整型元素组成的一维数组, pt的基类型是由4个整型元素组成的一维数组

数组指针的(物理)地址增量值以N-1维数组的长度为单位

类型一：指向数组元素的指针变量

```
#include <stdio.h>

int main()
{
    int a[3][4]={1,3,5,7,9,11,13,15,17,19,21,23};
    int *p; //p是int *型指针变量
    for(p=a[0];p<a[0]+12;p++) //使p依次指向下一个元素
    {
        if((p-a[0])%4==0) printf("\n"); //p移动4次后换行
        printf("%4d",*p); //输出p指向的元素的值
    }
    printf("\n");
    return 0;
}
```

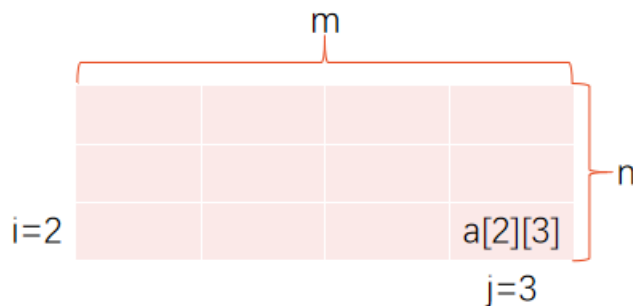


```
C:\WINDOWS\system32\cmd.exe
1 3 5 7
9 11 13 15
17 19 21 23
请按任意键继续...
```



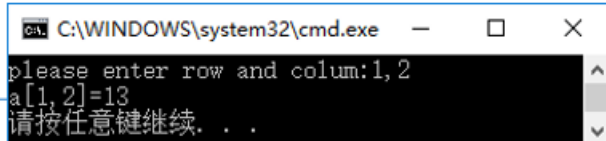
p是一个**int *型**(指向整型数据)的指针变量, 它可以指向一般的整型变量, 也可以指向整型的数组元素。每次使p值加1, 使p指向下一元素。

如果要输出某个指定的数值元素(例如a[2][3]), 则应事先计算该元素在数组中的**相对位置**(即相对于数组起始位置的相对位移量)。计算a[i][j]在数组中的相对位置的计算公式为: $i*m + j$, 其中, m为二维数组的列数(二维数组大小为n×m)。



类型二：指向由m个元素组成的一维数组的指针

```
#include <stdio.h>
int main()
{
    int a[3][4]={1,3,5,7,9,11,13,15,17,19,21,23};    //定义二维数组a并初始化
    int (*p)[4],i,j;    //指针变量p指向包含4个整型元素的一维数组
    p=a;    //p指向二维数组的0行
    printf("please enter row and colum:");
    scanf("%d,%d",&i,&j);    //输入要求输出的元素的行列号
    printf("a[%d,%d]=%d\n",i,j,*(*(p+i)+j));    //输出a[i][j]的值
    return 0;
}
```



```
C:\WINDOWS\system32\cmd.exe
please enter row and colum:1,2
a[1,2]=13
请按任意键继续. . .
```

```
#include <stdio.h>
int main()
{
    int a[4]={1,3,5,7};    //定义一维数组a，包含4个元素
    int (*p)[4];    //定义指向包含4个元素的一维数组的指针变量
    p=&a;    //使p指向一维数组
    printf("%d\n",(*p)[3]);    //输出a[3]，输出整数7
    return 0;
}
```

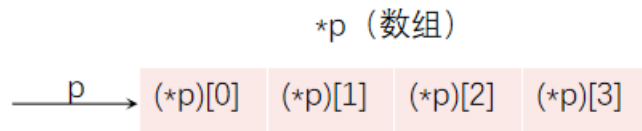


比较:

① int a[4]; (a有4个元素，每个元素都为整型)

② int (*p)[4];

第②种形式表示p所指的對象是有4个整型元素的一维数组。应该记住，此时p只能指向一个包含4个元素的一维数组，不能指向一维数组中的某一元素。p的值是该一维数组的起始地址。虽然这个地址（指纯地址）与该一维数组首元素的地址相同，但它们的基类型是不同的。



多维数组与指针的应用：多维数组可以看成是特殊的一维数组，当需要在函数之间进行多维数组的传递时，可以将多维数组进行降维处理，使之变为一维数组然后对降维后的一维数组进行相应的处理。

例6.10 有一个班，3个学生，各学4门功课，计算总平均成绩和第二个学生的平均成绩

```
#include <stdio.h>
float aver(float *,int n); //原型声明
void main( )
{
    float score[3][4]={{63,65,75,61},
                        {83,87,90,85},
                        {90,95,100,93}};

    printf("total average
score=%f\n",aver(*score,12));
    printf("second student average score
=%f",aver(score[1],4));
}
```

```
float aver(float *pdata,int n)
{
    int i;
    float average=0; //初始化为零

    for(i=0;i<n;i++)
        average+=pdata[i]; //累加和
    average=average/n; //求平均值
    return average; //将平均值返回
}
```

运行结果：

```
total average score=82.250000
second student average score=86.250000
```


6.4.1 指针数组：指针的集合，它的**每一个元素都是一个指针变量**，并且它们具有相同的存储类型和指向相同的数据类型。

指针数组的定义：

<存储类型> <数据类型> *指针数组名[元素个数];

`int *p[2];`

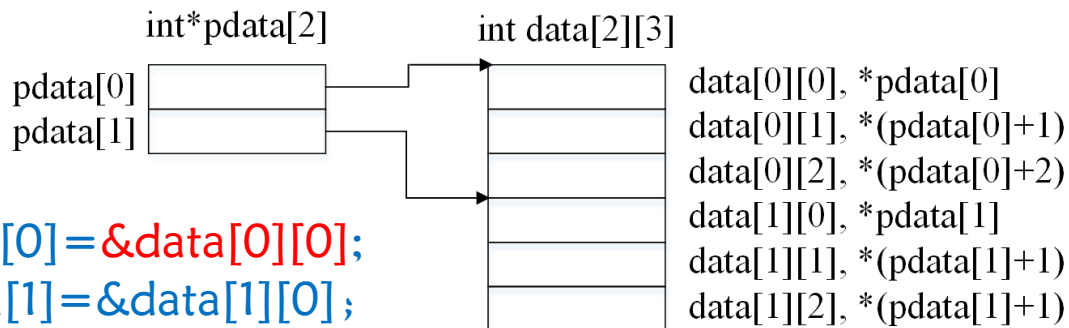
指针数组可以用来处理多维数组

`int data[2][3];`

`int *pdata[2];`

`pdata[0] = data[0];` 或 `pdata[0] = &data[0][0];`

`pdata[1] = data[1];` 或 `pdata[1] = &data[1][0];`



例6.15 用指针数组处理二维数组的数据

```
#include <stdio.h>
void main( )
{
```

```
    int data[2][3], *pdata[2];
    int i,j;
```

```
    for(i=0;i<2;i++)    //二维数组赋值
    for(j=0;j<3;j++)
```

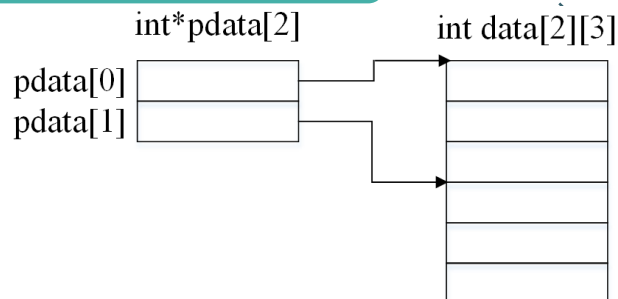
```
        data[i][j]=(i+1) *(j+1);
```

```
    pdata[0]=data[0]; pdata[1]=data[1];
```

```
    for(i=0;i<2;i++)
```

```
    for(j=0;j<3;j++,pdata[i]++)    //采用指针数组输出数组内容
        printf("data[%d][%d]:%-2d\n",i,j, *pdata[i]);
```

```
}
```



```
data[0][0], *pdata[0]
data[0][1], *(pdata[0]+1)
data[0][2], *(pdata[0]+2)
data[1][0], *pdata[1]
data[1][1], *(pdata[1]+1)
data[1][2], *(pdata[1]+1)
```

运行结果:

```
data[0][0]:1
data[0][1]:2
data[0][2]:3
data[1][0]:2
data[1][1]:4
data[1][2]:6
```

data[i][j]和*(data[i]+j), *(pdata[i]+j) 和pdata[i][j] 是意义相同的表示方法

6.4.1

指针数组

例6.15 指针数组和数组指针分别处理二维数组

```
#include<stdio.h>
void output1(int **app,int n);
void output2(int (*bpp)[3],int n);
void main( )
{
    int *ap[5];
    int (*bp)[3];
    int i,j;
    int arr[5][3]={1,2,3},{4,5,6},{7,8,9},{10,11,12},{13,14,15}};
    for(i=0;i<5;i++) ap[i]=arr[i];
    bp=arr;
    output1(ap, 5);    //以指针数组名作为实参
    output2(bp, 5);    //以数组指针名作为实参
}

void output1(int **app, int n)
{
    int i,j;
    printf("the array is:\n");
    for(i=0;i<n;i++)
    {
        for(j=0;j<3;j++)
            printf("%-5d", **app+i+j);
        printf("\n");
    }
}
```

bp →

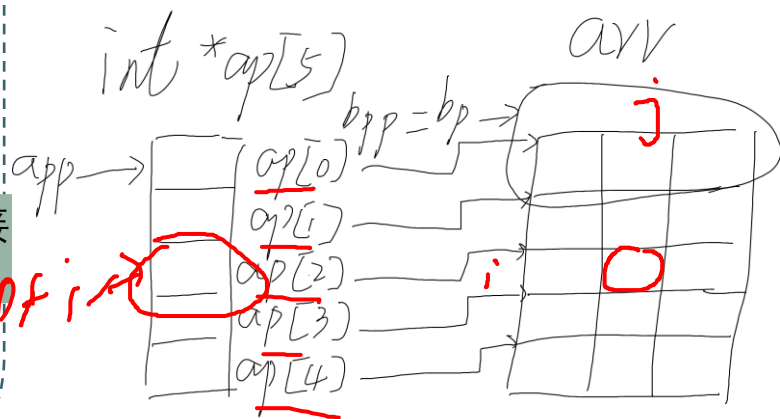
数组指针指向二维数组

二级指针作为形参接指针数组

```
void output2(int (*bpp)[3], int n)
{
    int i,j;
    printf("the array is:\n");
    for(i=0;i<n;i++)
    {
        for(j=0;j<3;j++)
            printf("%-5d", *(bpp+i+j));
        printf("\n");
    }
}
```

数组指针作为形参处理二维数组

运行结果：



```
10 11 12
13 14 15
```

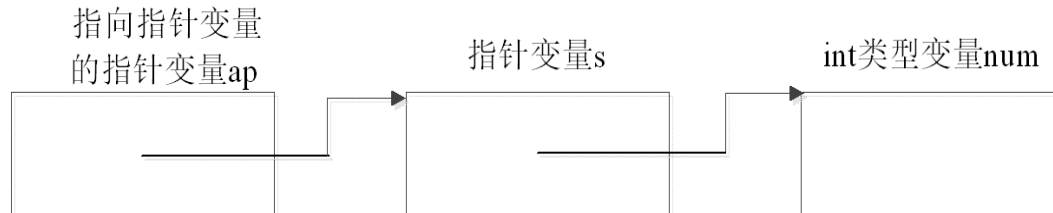
一个指针变量指向的变量仍然是一个指针变量，就构成指向指针变量的指针变量，简称**指向指针的指针**或**二级指针**

二级指针的声明形式：

存储类型 数据类型 **指针名；

目标变量的
数据类型

```
int **ap;  
int *s;  
int num;  
ap=&s;  
s=&num;  
num=100;
```



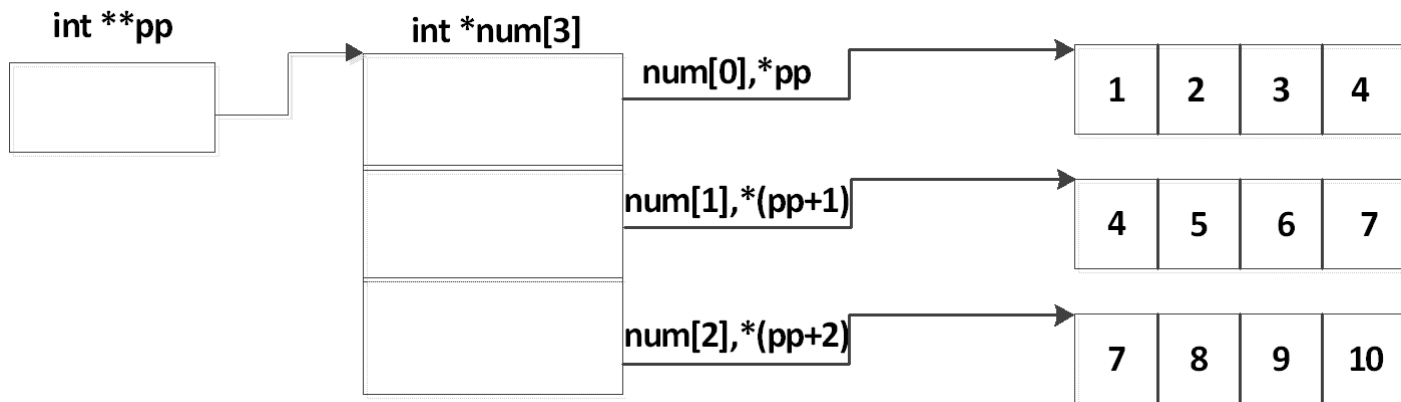
***ap**是取ap指向变量的内容，得到一个指针值即s中的内容

****ap**即 “* (*ap)” ，再取上述 “*ap” 指向变量的内容，得到num的值

num=100; ⇔ *s=100; ⇔ **ap=100;

程序中经常使用二级指针来处理指针数组

```
int arr[3][4]=  
{{1,2,3,4},{4,5,6,7},{7,8,9,10}};  
int *num[3];  
for (i=0; i<3;i++)  
    num[i]=arr[i];
```



pp指向指针数组num[]。pp的目标变量*pp就是num[0], *(pp+1)就是num[1], *(pp+2)就是num[2]。pp就是指向指针型数据的指针变量

例6.18 多级指针处理二维数组

```
#include <stdio.h>
int main( )
{
```

```
    int num[3][4]={0,1,2,3},{1,2,3,4},{2,3,4,5};
```

```
    int *p[3], **pp;
```

```
    int i, j;
```

```
    for (i=0; i<3; i++)
```

```
        p[i]=num[i];
```

```
    pp=p;    //使二级指针pp指向指针数组p的首元素地址
```

```
    for(i=0;i<3;i++)
```

```
    {
```

```
        for(j=0;j<4;j++)
```

```
            printf("%-3d", *(*(pp+i)+j));
```

```
            printf("\n");
```

```
        }
```

```
    }
```

运行结果:

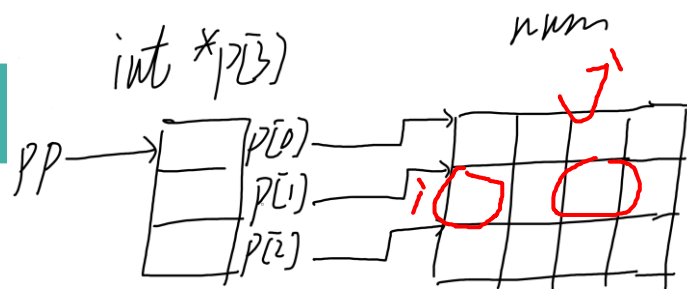
```
0 1 2 3
1 2 3 4
2 3 4 5
```

指针数组p存放数组num每行的首地址

借助二级指针输出数组元素

表达式等价

$*(*(pp+i)+j)$, $*(p[i]+j)$, $p[i][j]$,
 $*(*(num+i)+j)$, $*(num[i]+j)$, $num[i][j]$



例6.18 多级指针处理二维数组

```
#include <stdio.h>
void output(int **p, int n1, int n2);
int main( )
{ int num[3][4]={0,1,2,3},{1,2,3,4},{2,3,4,5}};
  int *p[3], **pp;
  for (int i=0; i<3; i++)
    p[i]=num[i];
  pp=p;
  output(pp, 3, 4); //调用函数
  return 0;
}

void output(int **p, int n1, int n2) //n1,n2为二级指针指向的数组
行列数
{
  int i, j;
  for(i=0;i<n1;i++)
  {
    for(j=0;j<n2;j++) printf("%-3d",*(*(p+i)+j));
    printf("\n");
  }
}
```

二级指针变量作为函数形参

0	1	2	3
1	2	3	4
2	3	4	5

二维数组、数组(的)指针、指针(的)数组、二级指针的联系

```
int a[3][4]; //二维数组a
```

```
int (*ap)[4]; //数组指针ap指向一包含4个元素的一维数组
```

```
int *pa[3]; //指针数组pa包含4个指针变量元素
```

```
int **p; //二级指针p
```

后面三个如何和二维数组关联？

ap = a; // **直接关联** ap就可以直接当二维数组使用了 ap[i][j]; *(ap[i]+j), *(*ap+i)+j)

for(int i=0;i<3;i++) pa[i] = a[i]; //指针数组与二维数组 **间接关联**

pa也可当二维数组使用 pa[i][j], *(pa[i]+j), *(*pa+i)+j)

p=pa; //直接关联, **p不能直接与二维数组名关联, 只能通过指针数组**

同上 p[i][j], *(p[i]+j), *(*p+i)+j)

所有这些关联的前提是一维数组与一级指针的互换性为基础。

6.5.0 指针变量作为函数参数

```
int main()
{
    void fun(int arr[], int n); //对fun函数的声明
    int array[10];             //定义array数组
    :
    fun(array,10);             //用数组名作函数的参数
    return 0;
}
void fun(int arr[], int n)     //定义fun函数    ≡
{
    :
}
```

```
void fun(int *arr, int n)     //定义fun函数
{
    :
}
```

array是实参数组名，arr为形参数组名。当用数组名作参数时，如果形参数组中各元素的值发生变化，实参数组元素的值随之变化。

在该函数被调用时，系统会在fun函数中建立一个指针变量arr，用来存放从主调函数传递过来的实参数组首元素的地址。

当arr接收了实参数组的首元素地址后，arr就指向实参数组首元素，也就是指向array[0]。

6.5.0 指针变量作为函数参数

如有一数组，想通过函数改变此数组中元素的值，实参与形参的对应关系有以下4种情况。

① 形参和实参都用数组名 ② 实参用数组名，形参用指针变量。

③ 实参形参都用指针变量 ④ 实参为指针变量，形参为数组名。

①

```
int main()
{
    int a[10];
    :
    f(a,10);
    :
}

int f(int x[], int n)
{
    :
}
```

②

```
int main()
{
    int a[10];
    :
    f(a,10);
    :
}

int f(int *x, int n)
{
    :
}
```

③

```
int main()
{
    int a[10];*p=a;
    :
    f(p,10);
    :
}

int f(int *x, int n)
{
    :
}
```

④

```
int main()
{
    int a[10];*p=a;
    :
    f(p,10);
    :
}

int f(int x[], int n)
{
    :
}
```

6.5.0 指针变量作为函数参数

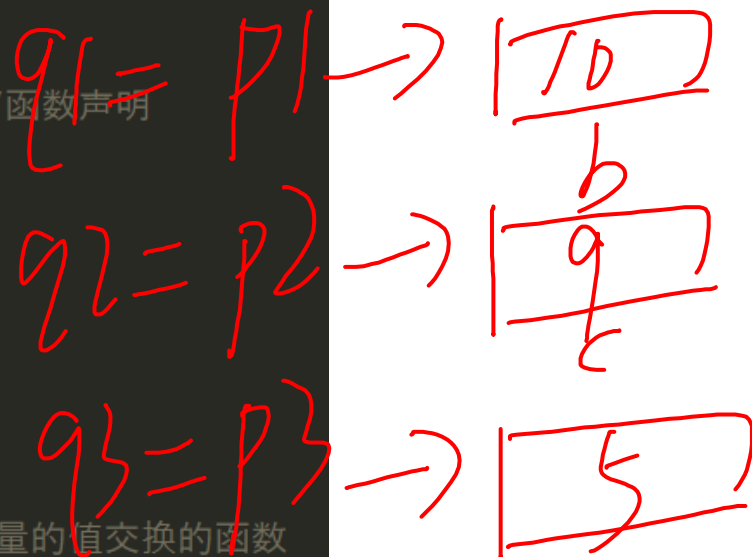
函数的调用可以（而且只可以）得到一个返回值（即函数值），而使用指针变量作参数，可以得到多个变化了的值。如果不用指针变量是难以做到这一点的。要善于利用指针法。如果想通过函数调用得到n个要改变的值，可以这样做：

- ①在主调函数中设n个变量，用n个指针变量指向它们；
- ②设计一个函数，有n个指针形参。在这个函数中改变这n个形参的值；
- ③在主调函数中调用这个函数，在调用时将这n个指针变量作实参，将它们的值，也就是相关变量的地址传给该函数的形参；
- ④在执行该函数的过程中，通过形参指针变量，改变它们所指向的n个变量的值；
- ⑤主调函数中就可以使用这些改变了值的变量。

```

1  #include <stdio.h>
2  int main()
3  {   void exchange(int *q1, int *q2, int *q3);    //函数声明
4      int a,b,c,*p1,*p2,*p3;
5      printf("please enter three numbers:");
6      scanf("%d,%d,%d",&a,&b,&c);
7      p1=&a;p2=&b;p3=&c;
8      exchange(p1,p2,p3);
9      printf("The order is:%d,%d,%d\n",a,b,c);
10     return 0;
11 }
12
13 void exchange(int *q1, int *q2, int *q3) //将3个变量的值交换的函数
14 {   void swap(int *pt1, int *pt2);    //函数声明
15     if(*q1<*q2) swap(q1,q2);          //如果a<b, 交换a和b的值
16     if(*q1<*q3) swap(q1,q3);          //如果a<c, 交换a和c的值
17     if(*q2<*q3) swap(q2,q3);          //如果b<c, 交换b和c的值
18 }
19
20 void swap(int *pt1, int *pt2)          //交换2个变量的值的函数
21 {   int temp;
22     temp=*pt1;                          //交换*pt1和*pt2变量的值
23     *pt1=*pt2;
24     *pt2=temp;
25 }

```



6.5.1 返回指针的函数：当函数的返回值是地址时，该函数就是指针型函数，其定义形式为：

<存储类型> <数据类型> *函数名(函数的形式参数及说明);

该函数本身的存储特性，分外部型或static型

返回值地址所在的内存空间中存储数据的数据类型

函数体内须有return语句，其后跟随的表达式结果值可以是变量的地址、数组首地址、已经定向的指针变量、结构变量地址、结构数组的首地址等



这些变量和数组须是全局或静态型，不能把被调用函数内的自动变量的地址和自动型数组的首地址作为指针函数的返回值

例6.20 求平方（错误例）

```
#include <stdio.h>
double *square(double);
int main()
{
    double num = 5.0;
    double *ptr = 0;
    ptr = square(num);
    printf("Num's square = %f \n", num*num);
    printf("Result = %f \n", *ptr);
    return 0;
}

double *square(double data)
{
    double result = 0.0;
    result = data*data;
    return &result;
}
```

永远不要从函数
中返回局部自动
变量的地址

运行结果：
Num's square = 25.000000

例6.21 模拟成绩检索系统，对学生成绩进行搜索，找出其中有不及格分数的学生学号和不及格课程号及分数。学生的成绩按照百分制进行记录，学生的学号为（1-10），课程编号为（1-4）。

问题分析与设计：

数据结构：建立学生成绩10*4二维数组，行代表不同学号，列代表不同课程。

函数原型：完成一行数据的搜索，搜索到不及格分数时，终止搜索，返回不及格分数的存储地址，以及其在数组中的列号

```
int *search( int (*pointer)[4], int *pm);
```

返回不及格分数的存储地址

该不及格分数在数组中的列号

6.5.1

返回指针的函数

```
#include <stdio.h>
/*声明查找函数*/
int *search( int (*pointer)[4], int *pm);
int main( )
{ int score[10][4]
={ {67,68,78,88},{90,69,66,79},{67,70,89,85},{65,76,69,70}
},{78,87,83,79},{88,70,48,57},{80,63,90,84},{67,48,70,84},
{92,90,77,70},{87,88,69,84} };
int *pt_1;
int row,column=0;
int *pm_1=&column;
for(row=0;row<10;row++)
{
    pt_1=search(score+row,pm_1);
    if(pt_1!=NULL) //出现不及格
    {
        printf("Stu[%d]'s No.%d course's score
is:",row+1,column);
        printf("%d\n",*pt_1);
        row--; //该行未扫描完，应继续扫描剩下列
    }
    else *pm_1=0; //表示该行已扫描完
}
}
```

逐行扫描的基础上，当发现有不及格现象的时候，转为逐个扫描

```
/*定义查找函数*/
//入口参数中：(*pointer)[4]是所查找的
一行，*pm用于存放列地址
int *search( int (*pointer)[4], int *pm)
{
    int i;    int *pt;
    pt=NULL;
    /*采用*pm间接传递了扫描列初址，当
    该生有多科不及格时为上次扫描之
    后一列*/
    for(i=*pm;i<4;i++)
    {
        if(*(*pointer+i)<60)
        {
            pt=*pointer+i;
            *pm=(i+1);
            break;
        }
    }
    return pt;
}
```

数组指针传递二维数组

运行结果：

Stu[6]'s No.3 course's score is:48
Stu[6]'s No.4 course's score is:57
Stu[8]'s No.2 course's score is:48

在C语言中，函数整体不能作为参数直接传递给另一个函数。尽管函数不是变量，但它却具有内存物理地址。函数的函数名和数组相似，**函数名表示该函数的存储首地址，即函数的执行入口地址。**

函数指针：存储函数的入口地址，可以通过该指针变量来调用它所指的函数。

指针所指函数返回值的数据类型

其定义格式 <存储类型> <数据类型> (* 函数指针名)(参数表) [= 函数名];

函数指针本身的存储类型，有auto型、static型和extern型

float add(float a, float b);

float (*pfun)(float ,float);
pfun = add;

或

float (*pfun)(float,float) = add;

函数指针进行定向操作的两种方式

先定义后用地址赋值语句

初始化操作

定义一个函数指针时，**至少应指明参数表内的参数个数和各参数的数据类型**

函数和指向它的函数指针应具有相同的数据类型，即函数返回值的数据类型与函数指针的数据类型要一致，否则，应该采用强制类型转换，两者的形参表必须完全相同

函数指针与数据指针的不同

- 数据指针指向数据区，而函数指针指向程序代码区
- 数据指针的取内容运算表达式 “* 数据指针名”，是访问该指针所指的数据，而函数指针的取内容运算表达式 “* 函数指针名”，是使程序控制转移到函数指针所指的函数目标代码模块首地址，执行该函数的函数体目标代码。

函数指针代替函数名的调用语句一般格式：

(*函数指针名)(参数表); ①

函数指针名(参数表); ②

float c;

c = pfun(3.5, 6.2);

c = (*pfun)(3.5, 6.2);

在实际应用中，在一个执行过程中可以调用不同函数时，函数的传递能体现出较大的优越性。

例6.22 编写函数实现积分

```
#include <stdio.h>
#include <math.h>
float integrate(float a , float b, int n, float (*f)(float));
float sin2(float x);
float f1(float x);
/*积分函数定义*/
/*a,b为积分边界, n为积分区间分割数, f指向被积分函数*/
float integrate(float a , float b, int n, float (*f)(float))
{
    float s,d;
    int i;
    d=(b-a)/n;          //微元精度d
    s=(*f)(a)*d;         //微元面积
    for( i=1;i<=n-1;i++)
        s=s+(*f)(a+i*d)*d; //微元面积相加
    return s;
}
```

```
/*定义被积函数*/
float sin2(float x){ return sin(x)*sin(x); }
float f1(float x){ return x*x+x/2; }
```

```
int main( )
```

```
{
    //定义指针指向函数sin2
    float (*p)( float ) = sin2;
    printf("%f \n",integrate(0,1,100,p));
    printf("%f",integrate(-1,2,100,f1));
    return 0;
}
```

函数指针变量
作为实参

函数名作为实参

运行结果:
0.269143
3.682950

例6.25 编写程序，用指针形式访问数组元素，实现矩阵转置功能

问题分析与设计：

模块划分：矩阵转置功能块：完成矩阵的转置。

数据结构：采用**数组指针**，以地址传递方式实现函数间二维数组（矩阵）传递。

函数原型：

```
void convert(int (*a)[], int (*at)[], int row , int col );
```

功能：矩阵的转置

形参：数组指针a传递转置前的矩阵，数组指针at返回转置后的矩阵数据
row和col分别为矩阵的行列数

6.6

综合举例

```
#include <stdio.h>
```

```
#define N1 3
```

```
#define N2 4
```

```
void convert(int (*a)[N2], int (*at)[N1], int row, int col);
```

宏定义行列大小

函数声明

```
int main( )
```

```
{
```

```
int arr1[N1][N2]={1,2,3,4,5,6,7,8,9,10,11,12};
```

```
int arr2[N2][N1];
```

转置后的目标矩阵

```
int (*p)[N2], (*pt)[N1];
```

定义数组指针

```
int i, j;
```

```
p=arr1; pt=arr2;
```

```
convert( p, pt, N1, N2);
```

函数调用

```
printf("the new array is:\n");
```

```
for(i=0; i<N2; i++)
```

```
{
```

```
for(j=0; j<N1; j++) printf("%-4d",*(pt+i+j));
```

```
printf("\n");
```

```
}
```

```
printf("the new array is:\n");
```

```
for(i=0; i<N2; i++)
```

```
{
```

```
for(j=0; j<N1; j++) printf("%-4d",arr2[i][j]);
```

```
printf("\n");
```

```
}
```

采用指针方法输出

采用数组方法输出

```
/**转置函数 convert()
```

```
**数组指针a和at指向转置前后的矩阵
```

```
**row和col为转置前矩阵的行列值**/
```

```
void convert(int (*a)[N2], int (*at)[N1], int row, int col)
```

```
{
```

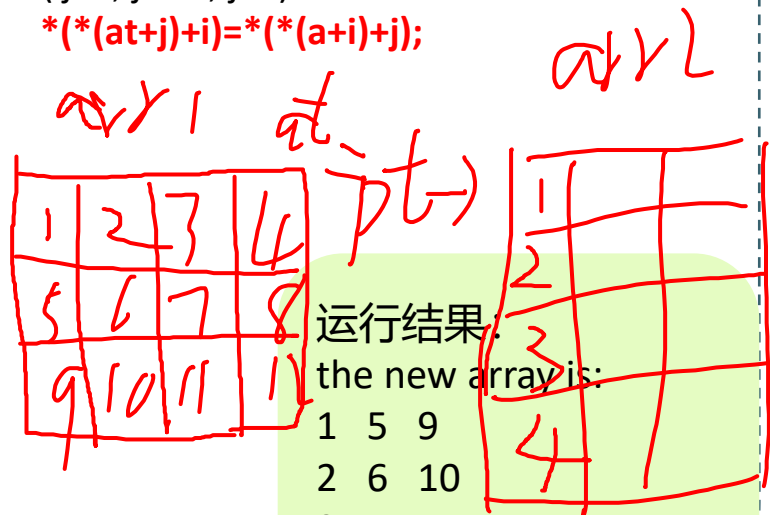
```
int i, j;
```

```
for( i=0; i<row; i++)
```

```
for( j=0; j<col; j++)
```

```
    *(*at+j+i)=*(*a+i+j);
```

```
}
```



运行结果:

the new array is:

1 5 9

2 6 10

3 7 11

4 8 12

例6.26 编写程序，对整型数组进行排序（冒泡法）**问题分析与设计：**

设计思路：定义函数指针以方便调用升降序逻辑函数，从而实现升序或降序的排序功能。

数据结构：采用一维数组存储待排序的数据，以地址传递方式实现函数间数组传递。

函数原型：

(1) int ascending(int a, int b);

功能：升序逻辑判断

形参：a,b为待比较的两个整数

返回：若 $a > b$, 值为1, 否则, 值为0

(2) int descending(int a, int b);

功能：降序逻辑判断

形参：a,b为待比较的两个整数

返回：若 $a < b$, 值为1, 否则, 值为0

(3) void exchange(int *a, int *b);

功能：两个整数的交换

形参：a,b为待交换的两个整数存储地址，通过地址传递方式，实现实参的交换

返回：无

(4) void sort(int a[], int n, int flag);

功能：按给定的升序或降序完成一维数组的排序

形参：a[]为指向待排序数组的指针，采用地址传递方式

n为数组的长度

flag为升降序标志

返回：无

6.6

综合举例

```
1  #include <stdio.h>
2  #include <stdbool.h>
3  /***** */
4  bool ascending( int, int );
5  bool descending( int, int );
6  void exchange( int *, int * );
7  void sort(int [], int, int );
8  int main( )
9  {
10     int num[10]={1,5,9,2,6,10,3,4,7,8};
11     int flag1; //升降序标志符
12     int i;
13     printf("Please decide direction: \n");
14     scanf("%d", &flag1);
15     sort(num,10,flag1);
16     printf("sorted array is:");
17     for( i=0; i<10; i++ )
18         printf( "%-4d", num[i] );
19     return 0;
20 }
21
```

```
22 void sort(int a[], int n, int flag )
23 {
24     bool ( *ad )( int , int );
25     int t, c;
26     if( flag )
27         ad=ascending;
28     else ad=descending;
29
30     for( t=0; t<n; t++ )
31         for( c=0; c<n-1; c++ )
32             if(( *ad )( a[c],a[c+1] ))
33                 exchange( &( a[c] ),&( a[c+1] ) );
34 }
35 bool ascending( int a, int b ){ return a>b;}
36 bool descending( int a, int b ){ return a<b;}
37 void exchange( int *a, int *b )
38 {
39     int temp;
40     temp= *a;
41     *a= *b;
42     *b= temp;
43 }
44
```


例6.27 编写程序，定义一个含有15个元素的数组，并编写函数分别完成以下操作：

- A) 调用C库函数的随机函数给数组元素赋0-99之间的随机值；
- B) 输出显示数组元素；
- C) 按顺序对每隔5个数求一个和数，并输出显示结果。

问题分析与设计：

设计思路：（1）通过调用包含在头文件stdlib.h之中的随机函数rand来生成随机数，生成0-(x-1)之间的随机数n的方法是： $n = \text{rand}() \% x$ ；

（2）两次输出显示可以采用调用一个函数的方法来实现。

模块划分：随机数产生模块、求和模块、显示模块

数据结构：采用一维数组存储生成的随机数序列和和数序列。

函数原型:

(1) void getrand(int *a, int n);

功能: 随机数序列生成

形参: a为指向数组存储空间的指针, n为数组长度

返回: 无

(2) void printarr(int *a, int n);

功能: 一维数据的输出

形参: a为指向数组的指针, n为数组长度

返回: 无

(3) void getsum(int *a,int *b,int n);

功能: 完成数据序列的分块求和

形参: a为指向原数组的指针, 采用地址传递方式

b为指向求和结果数组的指针

n为原数组的长度

返回: 无

```

#include <stdio.h>
#include <stdlib.h>
/*宏定义以方便修改*/
#define N1 15           定义数组长度
#define N2 5           求和间隔
void getrand(int *, int);      函数声明
void getsum(int *,int *,int);
void printarr(int *,int);
int main( )
{ /*x[N1]保存最初数组数据, w[N1/N2]保存求和数组数据*/
    int x[N1],w[N1/N2]={0};

    getrand(x,N1);           生成随机数组存入x[]

    printf("the number is:\n");
    printarr(x,N1);          输出x[]显示

    getsum(x,w,N1);          对x求和存入w

    printf("the sum number is:\n");
    printarr(w,N1/N2);       输出w[]显示
    return 0;
}

```

```
void getrand(int *a,int n)
```

```
{
```

```
    int i;
```

```
    for(i=0;i<n;i++)
```

```
        *(a+i)=rand()%100;  获得0-99随机数
```

```
}
```

```
void printarr(int *a, int n)
```

```
{
```

```
    int i;
```

```
    for(i=0;i<n;i++)
```

```
        printf("%4d",*(a+i));  宽度为4、右对齐显示数组
```

```
        printf("\n");
```

```
}
```

```
void getsum(int *a,int *b,int n)
```

```
{
```

```
    int i,j=0,sum=0;
```

```
    for(i=0;i<n;i++)
```

```
    {
```

```
        sum+=*(a+i);
```

```
        if((i+1)%5==0)
```

以每5个元素为单位进行求和

```
        {
```

```
            b[j]=sum; sum=0;    j++;
```

```
        }
```

```
}
```

求和并存入新的数组

随机数组生成函数

数组显示函数

数组分块求和函数

统计结果初始化为0

本章要掌握的内容有：

- 各种类型指针的定义形式与初始化方法和使用指针的方法；
- 指针允许运算的种类和运算方法；
- 如何用指针表示一维数组及二维数组，尤其注意指针和数组表现形式的互换性；
- 多级指针的概念、多级指针处理指针数组的方法；
- 区分数组指针与指针数组以及指针函数与函数指针并掌握其用法。